PYTHON PROGRAMMING - ASSIGNMENT 2 (CO2)

CONTROL FLOW, STRING OPERATIONS AND FUNCTIONS

Name:- Balachandru.P

Register Number:- 1912260301C

Slot:- C

Code & Course:- CSA08 - Python Programming

Faculty:- Pokuri Swetha

GitHub Repository Link:- _____

Contents: Q1 Decisions with many boundaries (6) | Q2 Loops that count attempts (4) | Q3 Vehicle registration validator (6) | Q4 Building long text (6) | Q5 Functions, parameters and scope (7) | Q6 Recursion and its limits (8) | Q7 Day-4 design challenge (13) | Total 50 marks

All programs were written as .py files and actually run; every output in this report is a real run on my computer (Python 3.10.12). Function-call counts and operation counts are given in tables next to the timings, because timings change from run to run but the counts do not.

Q1 - DECISIONS WITH MANY BOUNDARIES (6 Marks)

1. Aim

To write the electricity-subsidy rule twice, once with separate if statements and once with if-elif-else, to predict the decision for eight households before running anything, and to find out why the developer's version gave some households two subsidies.

2. Predictions made before running

The rule: up to 100 units gets the full subsidy; 101 to 200 units gets a partial subsidy only with a ration card; above 200 units there is no subsidy unless someone in the house is above 80 years old. I picked eight households on the boundaries. The predictions are written inside the program (the list called households) before any function runs:

Units	Ration card	Member above 80	My prediction	Why
100	No	No	FULL	boundary, 100 or less
100	Yes	No	FULL	boundary, card must not matter here
101	Yes	No	PARTIAL	first unit of the middle slab
101	No	No	NO	middle slab but no card
150	Yes	Yes	PARTIAL	elder only matters above 200
200	Yes	No	PARTIAL	last unit of middle slab
201	No	No	NO	above 200, nobody above 80
201	No	Yes	SENIOR	above 200 and elder

3. Program

File: Q1/q1_subsidy.py. It has the developer's separate-if version, my if-elif-else version, and two alternatives (a table of limits searched in a loop, and match-case).

```python
"""Q1 - Electricity subsidy: separate ifs vs if-elif-else (plus two alternatives)."""
import time


def subsidy_separate_ifs(units, ration_card, elder_above_80):
    """The developer's version: independent ifs, overlapping conditions."""
    results = []
    if units <= 100:
        results.append("FULL subsidy")
    if units <= 200 and ration_card:          # forgot the lower bound 101
        results.append("PARTIAL subsidy")
    if elder_above_80:                        # forgot that this only applies above 200
        results.append("SENIOR subsidy")
    return results if results else ["NO subsidy"]


def subsidy_elif(units, ration_card, elder_above_80):
    """Correct version: exactly one branch can run."""
    if units <= 100:
        return "FULL subsidy"
    elif 101 <= units <= 200:
        return "PARTIAL subsidy" if ration_card else "NO subsidy"
    else:
        return "SENIOR subsidy" if elder_above_80 else "NO subsidy"


SLABS = [(100, "FULL subsidy"), (200, "PARTIAL/NO"), (float("inf"), "ABOVE-200")]


def subsidy_table(units, ration_card, elder_above_80):
    """Alternative: list of (upper limit, decision) pairs searched in a loop."""
    for limit, kind in SLABS:
        if units <= limit:
            break
    if kind == "FULL subsidy":
        return kind
    if kind == "PARTIAL/NO":
        return "PARTIAL subsidy" if ration_card else "NO subsidy"
    return "SENIOR subsidy" if elder_above_80 else "NO subsidy"


def subsidy_match(units, ration_card, elder_above_80):
    """Alternative: match-case (Python 3.10+) with guards."""
    match (units, ration_card, elder_above_80):
        case (u, _, _) if u <= 100:
            return "FULL subsidy"
        case (u, True, _) if u <= 200:
            return "PARTIAL subsidy"
```

```
        case (u, False, _) if u <= 200:
            return "NO subsidy"
        case (_, _, True):
            return "SENIOR subsidy"
        case _:
            return "NO subsidy"


# (units, ration_card, elder_above_80, my prediction written BEFORE running)
households = [
    (100, False, False, "FULL subsidy"),     # boundary
    (100, True,  False, "FULL subsidy"),     # boundary, has card (trap for separate ifs)
    (101, True,  False, "PARTIAL subsidy"),  # boundary
    (101, False, False, "NO subsidy"),       # boundary, no card
    (150, True,  True,  "PARTIAL subsidy"),  # elder but not above 200 units
    (200, True,  False, "PARTIAL subsidy"),  # boundary
    (201, False, False, "NO subsidy"),       # boundary
    (201, False, True,  "SENIOR subsidy"),   # boundary + elder
]

print(f"{'units':>5} {'card':>6} {'elder':>6}  {'my prediction':<16} {'if-elif-else':<16} separate ifs")
ok = 0
double = 0
for units, card, elder, pred in households:
    fixed = subsidy_elif(units, card, elder)
    buggy = subsidy_separate_ifs(units, card, elder)
    ok += (fixed == pred)
    double += (len(buggy) > 1)
    flag = "  <-- TWO RESULTS" if len(buggy) > 1 else ""
    print(f"{units:>5} {str(card):>6} {str(elder):>6}  {pred:<16} {fixed:<16} {' + '.join(buggy)}{flag}")
print(f"\nPredictions matching if-elif-else: {ok}/{len(households)}")
print(f"Households where separate ifs gave more than one result: {double}/{len(households)}")

# check every combination 0..300 units for all four implementations
mismatch = 0
checked = 0
for u in range(0, 301):
    for c in (False, True):
        for e in (False, True):
            a = subsidy_elif(u, c, e)
            checked += 1
            if not (a == subsidy_table(u, c, e) == subsidy_match(u, c, e)):
                mismatch += 1
print(f"\nExhaustive check: {checked} combinations, mismatches between elif/table/match = {mismatch}")
multi = sum(len(subsidy_separate_ifs(u, c, e)) > 1 for u in range(301) for c in (False, True) for e in (False, True))
print(f"Separate-ifs version gave more than one result in {multi} of {checked} combinations")

# timing per decision (1,000,000 decisions each, worst-case slab = above 200)
```

```
N = 1_000_000
print(f"\nTiming {N:,} decisions (units=250, card=False, elder=True):")
for name, f in [("if-elif-else", subsidy_elif), ("table loop", subsidy_table), ("match-case", subsidy_match)]:
    best = 1e9
    for _ in range(3):
        t = time.perf_counter()
        for _ in range(N):
            f(250, False, True)
        best = min(best, time.perf_counter() - t)
    print(f"  {name:<14} {best:.3f} s  ({best / N * 1e9:.0f} ns per decision)")
```

4. Output

Real run of q1_subsidy.py (command: python3 q1_subsidy.py), Python 3.10.12.

```
Terminal - python3

$ python3 q1_subsidy.py
units   card  elder  my prediction    if-elif-else     separate ifs
  100  False  False  FULL subsidy     FULL subsidy     FULL subsidy
  100   True  False  FULL subsidy     FULL subsidy     FULL subsidy + PARTIAL subsidy  <-- TWO RESULTS
  101   True  False  PARTIAL subsidy  PARTIAL subsidy  PARTIAL subsidy
  101  False  False  NO subsidy       NO subsidy       NO subsidy
  150   True   True  PARTIAL subsidy  PARTIAL subsidy  PARTIAL subsidy + SENIOR subsidy  <-- TWO RESULTS
  200   True  False  PARTIAL subsidy  PARTIAL subsidy  PARTIAL subsidy
  201  False  False  NO subsidy       NO subsidy       NO subsidy
  201  False   True  SENIOR subsidy   SENIOR subsidy   SENIOR subsidy

Predictions matching if-elif-else: 8/8
Households where separate ifs gave more than one result: 2/8

Exhaustive check: 1204 combinations, mismatches between elif/table/match = 0
Separate-ifs version gave more than one result in 403 of 1204 combinations

Timing 1,000,000 decisions (units=250, card=False, elder=True):
  if-elif-else   0.080 s  (80 ns per decision)
  table loop     0.139 s  (139 ns per decision)
  match-case     0.187 s  (187 ns per decision)
```

5. Analysis

- Separate if statements are independent questions. Python checks every one of them, so if two conditions overlap, both run. In the developer's version I wrote the second condition as units <= 200 and ration_card (no lower bound) and the third as just elder_above_80 (no 'above 200' check). So a 100-unit household with a card got FULL + PARTIAL, and the 150-unit household with a card and an elder got PARTIAL + SENIOR. That is exactly the 'two subsidies' complaint, and my run shows it in 2 of the 8 households.
- if-elif-else is one chain: the moment one condition is true, Python runs that block and skips the rest, so exactly one result is possible. All 8 of my predictions matched the if-elif-else column (8/8).
- The boundaries 100, 101, 200 and 201 behave correctly because the middle slab is written as the chained comparison 101 <= units <= 200 and the first slab uses <=. I also checked every combination of units 0 to 300, card yes/no and elder yes/no (1,204 cases): the separate-if version gave more than one result in 403 of them, while the three correct versions never disagreed with each other (0 mismatches).

6. Design Decision

I used combined Boolean conditions inside one if-elif-else chain instead of nested ifs. Each slab is one branch of the chain, so the code reads like the rule table (<=100, 101-200, above 200), and the card/elder test sits inside its own slab. With nested ifs (if units > 100: if units <= 200: if card: ...) I would have needed three levels of indentation and it is easy to put an else under the wrong if. If the board changes a limit (say 200 to 250) I change one number in one place. The weak point of my choice is that a long chain with and/or gets hard to read if many more rules are added, and then the table version below is better.

7. Alternative Concept and Complexity Analysis

I wrote the rule two more ways and tested all of them against the same 1,204 inputs (0 mismatches). (a) A list of (upper limit, decision) pairs searched in a loop: SLABS = [(100, ...), (200, ...), (inf, ...)]. (b) match-case with guards (needs Python 3.10+). I timed 1,000,000 decisions for the last slab (units = 250):

Version	Time for 1,000,000	Per decision	Time for k slabs	Extra space

	decisions			
if-elif-else	0.080 s	80 ns	O(k) worst case	O(1)
table of (limit, decision) in a loop	0.139 s	139 ns	O(k)	O(k) for the table
match-case with guards	0.187 s	187 ns	O(k) (cases are tried in order)	O(1)

Table Q1-1: measured on my computer (best of 3 runs).

All three look at the slabs one after another, so the time per decision grows with the number of slabs k, O(k); for 3 slabs this is nothing. The table needs O(k) space to store the limits, the other two only O(1). The table is the easiest to change when slabs are added (add one pair) and it is the one I would use for 50 slabs, but for 3 slabs plain if-elif-else was the fastest and the simplest to read in my measurement.

8. Conclusion

The bug was in the structure, not the numbers: independent ifs cannot say 'exactly one of these'. The if-elif-else version matched all 8 predicted households and all 1,204 checked cases, and it was also the fastest of the three correct versions I measured.

Q2 - LOOPS THAT COUNT ATTEMPTS (4 Marks)

1. Aim

To write the bank-kiosk OTP attempt loop with while and with for + range(3), to use break, continue, pass and the loop-else clause, to answer the two questions, and to redo the design when the bank gives two extra attempts after a correct security answer.

Program

File: Q2/q2_otp.py. To keep the runs repeatable, the 'customer' is a list of typed entries instead of input(); an empty string is a blank entry. The correct OTP in the tests is 482913.

```
"""Q2 - OTP attempt loops: while, for+range(3), break/continue/pass, loop-else, changed rule, recursion."""
import sys
import time

CORRECT_OTP = "482913"
SECURITY_ANSWER = "chennai"


def make_reader(entries):
    it = iter(entries)
    return lambda prompt="": next(it, "")      # when the script runs out, behaves like a blank entry


# ---------- Version 1: while loop (original rule, 3 attempts) ----------
def kiosk_while(entries):
    read = make_reader(entries)
    attempts = 0                               # only real (non-blank) entries are counted
    screens = 0
    while attempts < 3:
        screens += 1
        otp = read("OTP: ").strip()
        if otp == "":
            continue                           # blank: ask again, do NOT count
        if otp == CORRECT_OTP:
            print(f"   screen {screens}: OTP accepted")
            break                              # success: leave the loop, else is skipped
        attempts += 1
        print(f"   screen {screens}: wrong OTP ({attempts}/3)")
    else:
        return f"LOCKED for 30 minutes after {attempts} wrong attempts ({screens} screens)"
    return f"ACCESS GRANTED on attempt {attempts + 1} ({screens} screens)"


# ---------- Version 2: for + range(3) ----------
def kiosk_for(entries):
    read = make_reader(entries)
    screens = 0
    for attempt in range(3):
        otp = read("OTP: ").strip()
        screens += 1
        while otp == "":                       # blank entries must be absorbed INSIDE one attempt
            otp = read("OTP: ").strip()
            screens += 1
        if otp == CORRECT_OTP:
            print(f"   attempt {attempt + 1}: OTP accepted")
            break
        print(f"   attempt {attempt + 1}: wrong OTP")
    else:
```

```
        return f"LOCKED for 30 minutes after 3 wrong attempts ({screens} screens)"
    return f"ACCESS GRANTED on attempt {attempt + 1} ({screens} screens)"


# ---------- the same for loop, but with a naive blank handling (shows why continue is wrong here) ----------
def kiosk_for_naive(entries):
    read = make_reader(entries)
    for attempt in range(3):
        otp = read("OTP: ").strip()
        if otp == "":
            continue                            # BUG: range already used up this attempt
        if otp == CORRECT_OTP:
            return f"ACCESS GRANTED on loop pass {attempt + 1}"
    else:
        return "LOCKED"


# ---------- bug: counter updated in the wrong place ----------
def kiosk_while_counter_bug(entries):
    read = make_reader(entries)
    attempts = 0
    while attempts < 3:
        attempts += 1                           # BUG: counted before the blank check
        otp = read("OTP: ").strip()
        if otp == "":
            continue
        if otp == CORRECT_OTP:
            break
    else:
        return f"LOCKED after counter reached {attempts}"
    return f"ACCESS GRANTED on attempt {attempts}"


# ---------- Changed condition: 2 extra attempts after a correct security answer ----------
def kiosk_while_extra(entries, security):
    read = make_reader(entries)
    allowed, attempts, extended = 3, 0, False
    while attempts < allowed:
        otp = read("OTP: ").strip()
        if otp == "":
            pass                                # pass: nothing to do for a blank, then continue below
            continue
        if otp == CORRECT_OTP:
            print(f"   OTP accepted on attempt {attempts + 1}")
            break
        attempts += 1
        print(f"   wrong OTP ({attempts}/{allowed})")
        if attempts == allowed and not extended:
```

```
            extended = True
            if security.strip().lower() == SECURITY_ANSWER:
                allowed += 2                    # only the while condition's limit changes
                print(f"   security answer correct -> allowed raised to {allowed}")
            else:
                print("   security answer wrong -> no extra attempts")
    else:
        return f"LOCKED for 30 minutes after {attempts} wrong attempts"
    return f"ACCESS GRANTED on attempt {attempts + 1}"


def kiosk_for_extra(entries, security):
    read = make_reader(entries)
    def one_round(n):
        for i in range(n):
            otp = read("OTP: ").strip()
            while otp == "":
                otp = read("OTP: ").strip()
            if otp == CORRECT_OTP:
                return i + 1
            print(f"   wrong OTP")
        return 0
    got = one_round(3)
    if got:
        return f"ACCESS GRANTED on attempt {got}"
    if security.strip().lower() == SECURITY_ANSWER:
        print("   security answer correct -> a second for loop of 2 attempts")
        got = one_round(2)
        if got:
            return f"ACCESS GRANTED on attempt {3 + got}"
    return "LOCKED for 30 minutes"


# ---------- Alternative: recursion ----------
depth_seen = 0
def kiosk_recursive(read, remaining, depth=1):
    global depth_seen
    depth_seen = max(depth_seen, depth)
    if remaining == 0:
        return "LOCKED"
    otp = read("OTP: ").strip()
    if otp == "":
        return kiosk_recursive(read, remaining, depth + 1)     # blank: same remaining
    if otp == CORRECT_OTP:
        return "GRANTED"
    return kiosk_recursive(read, remaining - 1, depth + 1)
```

```python
if __name__ == "__main__":
    scenarios = [
        ("S1 correct on 2nd try, with a blank in between", ["111111", "", CORRECT_OTP]),
        ("S2 three wrong entries",                          ["111111", "222222", "333333"]),
        ("S3 blank, blank, wrong, correct",                 ["", "", "999999", CORRECT_OTP]),
        ("S4 three blanks first, then the correct OTP",     ["", "", "", CORRECT_OTP]),
    ]
    print("=== PART 1: original rule (3 attempts, blank does not count) ===")
    for name, entries in scenarios:
        print(f"\n{name}  -> entries {entries}")
        print(" while:", kiosk_while(entries))
        print(" for  :", kiosk_for(entries))
    print("\n=== PART 2: why the naive for-loop with continue is wrong (scenario S4) ===")
    print(" for + continue :", kiosk_for_naive(["", "", "", CORRECT_OTP]))
    print(" for + inner while:", kiosk_for(["", "", "", CORRECT_OTP]))
    print("\n=== PART 3: counter updated in the wrong place (scenario S4) ===")
    print(" counter before blank check:", kiosk_while_counter_bug(["", "", "", CORRECT_OTP]))
    print(" counter after blank check :", kiosk_while(["", "", "", CORRECT_OTP]))

    print("\n=== PART 4: changed condition (2 extra attempts if security answer is correct) ===")
    cases = [
        ("C1 3 wrong, security right, correct on 5th", ["1", "2", "3", "4", CORRECT_OTP], "Chennai"),
        ("C2 3 wrong, security right, 2 more wrong",   ["1", "2", "3", "4", "5"],          "Chennai"),
        ("C3 3 wrong, security WRONG",                 ["1", "2", "3", CORRECT_OTP],       "Madurai"),
        ("C4 correct on 1st try",                      [CORRECT_OTP],                      "-"),
    ]
    for name, entries, sec in cases:
        print(f"\n{name}")
        print(" while:", kiosk_while_extra(entries, sec))
        print(" for  :", kiosk_for_extra(entries, sec))

    print("\n=== PART 5: recursion alternative ===")
    for name, entries in scenarios[:2]:
        depth_seen = 0
        print(f" {name}: {kiosk_recursive(make_reader(entries), 3)}  (deepest call depth {depth_seen})")
    print("\n Cost as the number of attempts a grows (all entries wrong):")
    print(f" sys.getrecursionlimit() = {sys.getrecursionlimit()}")
    print(f" {'a':>7} {'loop time (s)':>14} {'recursive time (s)':>19} {'max depth':>10}")
    for a in (100, 500, 900, 5000):
        wrong = lambda prompt="": "000000"
        t = time.perf_counter()
        att = 0
        while att < a:
            wrong(); att += 1
        tl = time.perf_counter() - t
        depth_seen = 0
        t = time.perf_counter()
        try:
            kiosk_recursive(wrong, a)
            tr = f"{time.perf_counter() - t:.6f}"
        except RecursionError:
            tr = "RecursionError"
        print(f" {a:>7} {tl:>14.6f} {tr:>19} {depth_seen:>10}")
```

Output

Real run of q2_otp.py (command: python3 q2_otp.py), Python 3.10.12.

Terminal - python3

```
$ python3 q2_otp.py
=== PART 1: original rule (3 attempts, blank does not count) ===

S1 correct on 2nd try, with a blank in between  -> entries ['111111', '', '482913']
   screen 1: wrong OTP (1/3)
   screen 3: OTP accepted
 while: ACCESS GRANTED on attempt 2 (3 screens)
   attempt 1: wrong OTP
   attempt 2: OTP accepted
 for  : ACCESS GRANTED on attempt 2 (3 screens)

S2 three wrong entries  -> entries ['111111', '222222', '333333']
   screen 1: wrong OTP (1/3)
   screen 2: wrong OTP (2/3)
   screen 3: wrong OTP (3/3)
 while: LOCKED for 30 minutes after 3 wrong attempts (3 screens)
   attempt 1: wrong OTP
   attempt 2: wrong OTP
   attempt 3: wrong OTP
 for  : LOCKED for 30 minutes after 3 wrong attempts (3 screens)

S3 blank, blank, wrong, correct  -> entries ['', '', '999999', '482913']
   screen 3: wrong OTP (1/3)
   screen 4: OTP accepted
 while: ACCESS GRANTED on attempt 2 (4 screens)
   attempt 1: wrong OTP
   attempt 2: OTP accepted
 for  : ACCESS GRANTED on attempt 2 (4 screens)

S4 three blanks first, then the correct OTP  -> entries ['', '', '', '482913']
   screen 4: OTP accepted
 while: ACCESS GRANTED on attempt 1 (4 screens)
   attempt 1: OTP accepted
 for  : ACCESS GRANTED on attempt 1 (4 screens)

=== PART 2: why the naive for-loop with continue is wrong (scenario S4) ===
 for + continue : LOCKED
   attempt 1: OTP accepted
 for + inner while: ACCESS GRANTED on attempt 1 (4 screens)

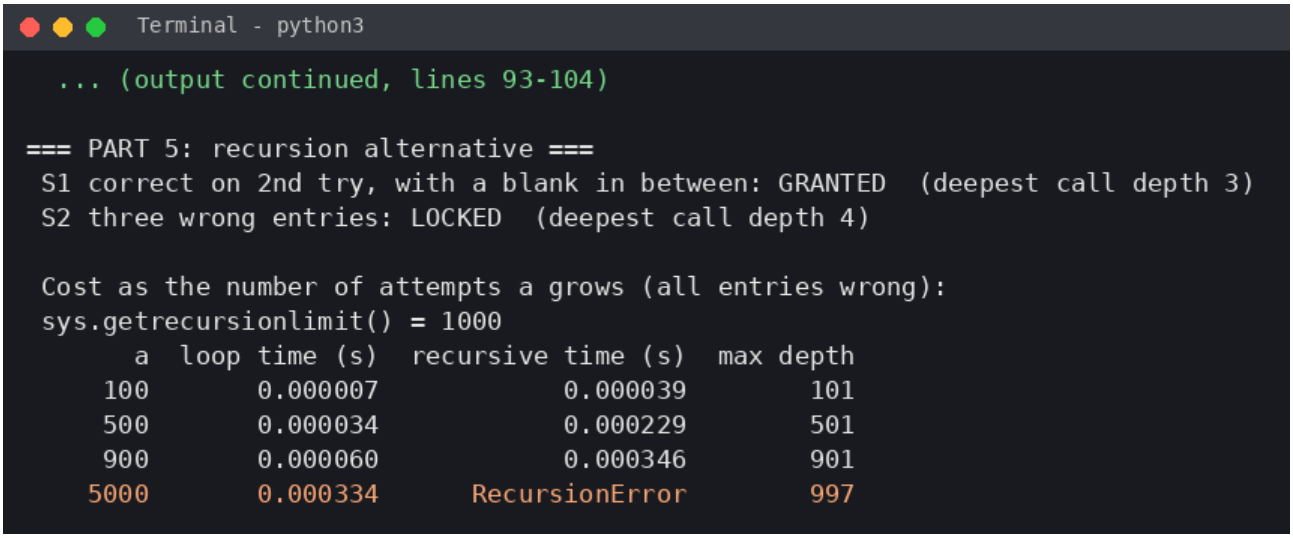
=== PART 3: counter updated in the wrong place (scenario S4) ===
 counter before blank check: LOCKED after counter reached 3
   screen 4: OTP accepted
 counter after blank check : ACCESS GRANTED on attempt 1 (4 screens)

=== PART 4: changed condition (2 extra attempts if security answer is correct) ===
```

```
Terminal - python3
  ... (output continued, lines 47-92)
C1 3 wrong, security right, correct on 5th
   wrong OTP (1/3)
   wrong OTP (2/3)
   wrong OTP (3/3)
   security answer correct -> allowed raised to 5
   wrong OTP (4/5)
   OTP accepted on attempt 5
 while: ACCESS GRANTED on attempt 5
   wrong OTP
   wrong OTP
   wrong OTP
   security answer correct -> a second for loop of 2 attempts
   wrong OTP
 for  : ACCESS GRANTED on attempt 5

C2 3 wrong, security right, 2 more wrong
   wrong OTP (1/3)
   wrong OTP (2/3)
   wrong OTP (3/3)
   security answer correct -> allowed raised to 5
   wrong OTP (4/5)
   wrong OTP (5/5)
 while: LOCKED for 30 minutes after 5 wrong attempts
   wrong OTP
   wrong OTP
   wrong OTP
   security answer correct -> a second for loop of 2 attempts
   wrong OTP
   wrong OTP
 for  : LOCKED for 30 minutes

C3 3 wrong, security WRONG
   wrong OTP (1/3)
   wrong OTP (2/3)
   wrong OTP (3/3)
   security answer wrong -> no extra attempts
 while: LOCKED for 30 minutes after 3 wrong attempts
   wrong OTP
   wrong OTP
   wrong OTP
 for  : LOCKED for 30 minutes
```



4. Analysis

- break leaves the loop the moment the OTP matches (scenario S1: accepted on screen 3). Because break skips the loop's else clause, the lock only happens when the loop finishes without a break. continue is used for a blank entry: it jumps back to the top without adding to the attempt counter. pass does nothing; I used it in the changed-condition version in the blank-entry branch, before the continue (the loop would work without it, so there pass is for readability, not logic). The loop-else is where the account lock lives: in S2 and C2 it ran after three (or five) wrong entries.
- When is while more natural? When the number of repetitions is not fixed before the loop starts. Here it is 3, but the changed rule can make it 5, and blank entries mean the number of screens (S4 needed 4 screens for 1 attempt) is not the number of attempts. A loop 'while attempts < allowed' just checks a variable each time, so changing allowed in the middle works. A for with range(3) fixes the count at the start; the blank-entry case needs an inner while (my kiosk_for) because range has already used up the pass.
- Bug if the counter is updated in the wrong place: I tested it. If attempts += 1 is placed before the blank check, a blank entry costs an attempt. With scenario S4 (three blank entries and then the correct OTP) that version printed 'LOCKED after counter reached 3', while the correct version printed 'ACCESS GRANTED on attempt 1 (4 screens)'. The same mistake happens in the for loop if the blank handling is just continue: kiosk_for_naive also printed LOCKED for S4, because range(3) was used up by the blank screens.

5. Changed Condition

Rule: after the three wrong attempts, if the customer answers a security question correctly, they get two more attempts. In the while design the only change is to raise the limit once: allowed = 5 when attempts == allowed and the security answer is right (the condition attempts < allowed then lets the loop continue). In my four test cases: C1 (security right, correct OTP on the 5th attempt) was granted; C2 (security right, two more wrong) was LOCKED after 5 wrong attempts; C3 (security answer wrong) was LOCKED after 3; C4 (correct first time) never reached the question. The for version needed a second for loop of range(2) after the first loop, so it gave the same decisions but needed more code. The while design carried the new rule with one extra if, so it handles the changed condition better.

6. Alternative Concept and Complexity Analysis

A recursive version, kiosk_recursive(read, remaining), asks for one OTP and calls itself for the next try (same remaining after a blank, remaining - 1 after a wrong one); the base case remaining == 0 means LOCKED. It gave the same results as the loops for S1 and S2 (deepest call depth 3 and 4). I measured it with all-wrong entries as the number of attempts a grows:

Attempts a	Loop time (s)	Recursive time (s)	Max call depth reached
100	0.000007	0.000039	101
500	0.000034	0.000229	501
900	0.000060	0.000346	901
5000	0.000334	RecursionError	997

Table Q2-1: sys.getrecursionlimit() = 1000 in my run.

Both do O(a) work for a attempts (each attempt is one read and one comparison). The difference is space: the loop keeps only a couple of variables, O(1), while each recursive call stays on the stack until the end, O(a). At a = 5000 the recursive version stopped with RecursionError at depth 997. For three attempts none of this matters, but the loop is safer and just as clear here.

7. Conclusion

Both loop designs give the same answer for the original rule. The changed condition and the counter-placement test showed the while loop is the better fit, because it can change its limit while running and it counts only what should be counted. The loop-else made the lock-out happen only when break never did.

Q3 - VEHICLE REGISTRATION NUMBER VALIDATOR (6 Marks)

1. Aim

To normalise typed vehicle numbers, check the pattern (2 letters, 2 digits, 1 or 2 letters, 4 digits) and print it as 'TN 09 AB 1234', using only string operations and no regular expressions.

Program

File: Q3/q3_registration.py. TODO 1 (normalise) and TODO 2 (is_valid) are completed; format_receipt and the regex comparison are extra.

```python
"""Q3 - Vehicle registration number validator using only string operations (no regex),
plus the regex alternative for comparison."""
import re
import time


def normalise(text):
    """'tn 09 ab 1234' -> 'TN09AB1234' (spaces and hyphens removed, upper case)."""
    # TODO 1 done: remove spaces and hyphens, then upper-case
    return text.replace(" ", "").replace("-", "").upper()


def is_valid(reg):
    """Valid form: 2 letters, 2 digits, 1 or 2 letters, 4 digits (e.g. TN09AB1234)."""
    # TODO 2 done: length 9 or 10, then check each slice with isalpha()/isdigit()
    if len(reg) != 9 and len(reg) != 10:
        return False
    state = reg[:2]
    district = reg[2:4]
    series = reg[4:-4]          # 1 letter for length 9, 2 letters for length 10
    number = reg[-4:]
    return state.isalpha() and district.isdigit() and series.isalpha() and number.isdigit()


def format_receipt(reg):
    """'TN09AB1234' -> 'TN 09 AB 1234' (only for a valid, normalised number)."""
    return " ".join([reg[:2], reg[2:4], reg[4:-4], reg[-4:]])


def is_valid_regex(reg):
    return re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg) is not None


if __name__ == "__main__":
    print("=== Complete the Code (given test list) ===")
    for raw in ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"]:
        reg = normalise(raw)
        print(f"{raw:15} -> {reg:11} {'valid' if is_valid(reg) else 'INVALID'}")

    print("\n=== Six designed inputs: prediction vs result ===")
    designed = [
        ("tn 09 ab 1234",  "TN09AB1234",  True),
        ("Tn09Ab1234",     "TN09AB1234",  True),
        ("TN-09-AB-1234",  "TN09AB1234",  True),
        ("ka 01 m 0007",   "KA01M0007",   True),
        ("TN 09 ABC 1234", "TN09ABC1234", False),   # 3 letters in series -> length 11
        ("T9 09 AB 1234",  "T909AB1234",  False),   # digit inside the state code
    ]
```

```
    right = 0
    for raw, pred_norm, pred_valid in designed:
        reg = normalise(raw)
        ok = is_valid(reg)
        match = (reg == pred_norm and ok == pred_valid)
        right += match
        shown = format_receipt(reg) if ok else "-"
        print(f"{raw:16} predicted {pred_norm:12}{str(pred_valid):6} | got {reg:12}{str(ok):6} receipt: {shown:15} {'MATCH' if match else 'DIFFERENT'}")
    print(f"Predictions matched: {right}/{len(designed)}")

    print("\n=== Inquiry: immutability, upper(), slicing ===")
    text = "tn09ab1234"
    print("text            :", text, "id", id(text))
    up = text.upper()
    print("text.upper()    :", up, "id", id(up))
    print("text afterwards :", text, "(unchanged)")
    try:
        text[0] = "T"
    except TypeError as e:
        print("text[0] = 'T'   -> TypeError:", e)
    for s in ("TN09A1234", "TN09AB1234"):
        print(f"{s!r}  len={len(s)}  s[-4:]={s[-4:]!r}  s[4:-4]={s[4:-4]!r}")

    print("\n=== Edge case: str methods vs regex ===")
    tricky = ["TN09AB1234", "TN\u0966\u0969AB1234", "T\u00c9" + "09AB1234", "TN\uff10\uff19AB1234"]
    for t in tricky:
        print(f"{ascii(t):32} string-methods: {is_valid(t)!s:6} regex: {is_valid_regex(t)}")

    print("\n=== Agreement test on generated strings ===")
    import random
    random.seed(7)
    alphabet = "AB09 -x"
    total, diff = 20000, 0
    for _ in range(total):
        s = normalise("".join(random.choice(alphabet) for _ in range(random.randint(7, 12))))
        if is_valid(s) != is_valid_regex(s):
            diff += 1
    # plus a structured set that is valid half the time
    for _ in range(total):
        s = (random.choice(["TN", "T9", "KA"]) + random.choice(["09", "0A", "22"]) +
             random.choice(["A", "AB", "A1", "ABC"]) + random.choice(["1234", "12a4", "123"]))
        total += 1
        if is_valid(s) != is_valid_regex(s):
            diff += 1
    print(f"{total} strings tested, disagreements between the two validators: {diff}")

    print("\n=== Timing: 200,000 validations of a 10-character number ===")
    N = 200_000
```

```
    for name, f in [("string methods", is_valid), ("re.fullmatch", is_valid_regex)]:
        best = 9
        for _ in range(3):
            t = time.perf_counter()
            for _ in range(N):
                f("TN09AB1234")
            best = min(best, time.perf_counter() - t)
        print(f"{name:15} {best:.3f} s  ({best / N * 1e9:.0f} ns each)")
    print("Cost of normalise + is_valid as the typed text gets longer (normalise is O(L)):")
    for L in (10, 1000, 100000, 1000000):
        s = "tn 09 ab 1234".ljust(L, "x")
        reps = 100 if L <= 100000 else 10
        t = time.perf_counter()
        for _ in range(reps):
            is_valid(normalise(s))
        print(f"  L={L:>8}: {(time.perf_counter() - t) / reps * 1e6:10.2f} us per call")
```

Output

Real run of q3_registration.py (command: python3 q3_registration.py), Python 3.10.12.

```
Terminal - python3
$ python3 q3_registration.py
=== Complete the Code (given test list) ===
tn 09 ab 1234    -> TN09AB1234  valid
KA-01-M-0007     -> KA01M0007   valid
TN9AB1234        -> TN9AB1234   INVALID
TN 22 1234 AB    -> TN221234AB  INVALID

=== Six designed inputs: prediction vs result ===
tn 09 ab 1234    predicted TN09AB1234  True   | got TN09AB1234  True   receipt: TN 09 AB 1234   MATCH
Tn09Ab1234       predicted TN09AB1234  True   | got TN09AB1234  True   receipt: TN 09 AB 1234   MATCH
TN-09-AB-1234    predicted TN09AB1234  True   | got TN09AB1234  True   receipt: TN 09 AB 1234   MATCH
ka 01 m 0007     predicted KA01M0007   True   | got KA01M0007   True   receipt: KA 01 M 0007    MATCH
TN 09 ABC 1234   predicted TN09ABC1234 False  | got TN09ABC1234 False  receipt: -               MATCH
T9 09 AB 1234    predicted T909AB1234  False  | got T909AB1234  False  receipt: -               MATCH
Predictions matched: 6/6

=== Inquiry: immutability, upper(), slicing ===
text            : tn09ab1234 id 139687522540272
text.upper()    : TN09AB1234 id 139687522545968
text afterwards : tn09ab1234 (unchanged)
text[0] = 'T'   -> TypeError: 'str' object does not support item assignment
'TN09A1234'  len=9  s[-4:]='1234'  s[4:-4]='A'
'TN09AB1234'  len=10  s[-4:]='1234'  s[4:-4]='AB'

=== Edge case: str methods vs regex ===
'TN09AB1234'                    string-methods: True   regex: True
'TN\u0966\u0969AB1234'          string-methods: True   regex: False
'T\xc909AB1234'                 string-methods: True   regex: False
'TN\uff10\uff19AB1234'          string-methods: True   regex: False

=== Agreement test on generated strings ===
40000 strings tested, disagreements between the two validators: 0

=== Timing: 200,000 validations of a 10-character number ===
string methods  0.055 s  (276 ns each)
re.fullmatch    0.078 s  (388 ns each)
Cost of normalise + is_valid as the typed text gets longer (normalise is O(L)):
  L=      10:       0.53 us per call
  L=    1000:       1.16 us per call
  L=  100000:     103.35 us per call
  L= 1000000:    1374.18 us per call
```

The Complete-the-Code part matches the expected output exactly: tn 09 ab 1234 -> TN09AB1234 valid; KA-01-M-0007 -> KA01M0007 valid; TN9AB1234 -> TN9AB1234 INVALID; TN 22 1234 AB -> TN221234AB INVALID.

4. Predictions for my six designed inputs

Input	Predicted normal form	Predicted decision	Result of the run
tn 09 ab 1234	TN09AB1234	valid	match
Tn09Ab1234	TN09AB1234	valid	match
TN-09-AB-1234	TN09AB1234	valid	match
ka 01 m 0007	KA01M0007	valid	match
TN 09 ABC 1234	TN09ABC1234	INVALID (3 series letters, length 11)	match
T9 09 AB 1234	T909AB1234	INVALID (digit in state code)	match

All six predictions matched (6/6). The two rejected inputs are the last two. Valid ones were printed for the receipt as TN 09 AB 1234 and KA 01 M 0007 by format_receipt().

5. Inquiry - immutability and slicing

- Why strings are immutable: a string never changes after it is created, so the same string object can safely be shared by many variables, used as a dictionary key (its hash never changes) and cached. My run shows it: text[0] = 'T' gives TypeError: 'str' object does not support item assignment.
- Why text.upper() does not change text: since a string cannot change, upper() builds a NEW string. The run printed two different ids (139970057149168 for text and 139970057154992 for the result) and text afterwards was still 'tn09ab1234'. That is why normalise() returns the result instead of expecting text to change.
- s[-4:] and s[4:-4]: for the 9-character number TN09A1234, s[-4:] = '1234' and s[4:-4] = 'A'; for the 10-character TN09AB1234, s[-4:] = '1234' and s[4:-4] = 'AB'. The same two slices work for both lengths because they count from the end: the last four characters are always the digits, and whatever is between position 4 and the last four is the series (1 or 2 letters). That is why is_valid only needs to check that the length is 9 or 10.

One thing I found while testing: isdigit() and isalpha() also accept non-English characters. A number with Devanagari digits and one with full-width digits both passed my string-method validator but failed the regex [0-9] (see the 'Edge case' lines in the output). To be strict I would add .isascii() to each check. I did not add it to the given function because the question says to use isalpha() and isdigit(), but it is a real limitation.

6. Alternative Concept and Complexity Analysis

With a regular expression the whole validation is one line: re.fullmatch(r'[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}', reg). I ran both validators on 40,000 generated strings (random ones, and ones built to be nearly valid) and got 0 disagreements. Timing 200,000 validations of TN09AB1234: string methods 0.054 s (276 ns each) and re.fullmatch 0.078 s (388 ns each).

Length L of typed text	normalise + is_valid time per call
10	0.53 us
1000	1.16 us
100000	103.35 us
1000000	1374.18 us

Table Q3-1: normalise is O(L) (replace and upper read every character); the length check in is_valid then rejects a long string at once.

Both approaches are O(L). Readability: the regex states the pattern in one line and is easier to change if the pattern changes (for example {1,3} for the series), but it is harder for a beginner to read. The slicing version is longer but each line says what it checks, and I can print which part failed. For this assignment (the rule is 'only string operations') slicing is the answer; in real code I would probably use the regex.

7. Conclusion

replace, upper, slicing with negative indexes, isalpha and isdigit were enough to normalise, validate and format every input. All predictions matched, and the regex comparison and the timing both showed the two methods behave the same and grow in the same O(L) way.

Q4 - BUILDING LONG TEXT: + OR join()? (6 Marks)

1. Aim

To build the monthly statement text three ways (repeated +=, join(), an f-string line each), to say which is faster before measuring, and to measure 1,000, 10,000 and 200,000 lines with time.perf_counter().

Program

File: Q4/q4_build_text.py. TODO 1 (build_join) is completed; f-string, StringIO, file and 'defeated +=' versions are extra.

```
"""Q4 - Building long text: += vs join() vs f-string lines; measured at 1,000 / 10,000 / 200,000 lines."""
import io
import sys
import time
import tracemalloc

items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]


def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text


def build_join(parts):
    # TODO 1 done: single str.join() call
    return ";".join(parts) + ";"


def build_fstring(parts):
    text = ""
    for p in parts:
        text += f"{p};"
    return text


def build_plus_defeated(parts):
    """Same += logic, but a second reference to the string stops CPython's in-place trick."""
    text = ""
    for p in parts:
        keep = text            # second reference: refcount is now 2
        text += p + ";"
    return text


def build_stringio(parts):
    buf = io.StringIO()
    for p in parts:
        buf.write(p)
        buf.write(";")
    return buf.getvalue()


def build_file(parts, path="statement_out.txt"):
    with open(path, "w") as f:
        for p in parts:
```

```
            f.write(p + ";")
    return path


def timed(fn, data):
    t = time.perf_counter()
    r = fn(data)
    return time.perf_counter() - t, r


if __name__ == "__main__":
    # ---- the exact given test ----
    start = time.perf_counter(); a = build_plus(items); t_plus = time.perf_counter() - start
    start = time.perf_counter(); b = build_join(items); t_join = time.perf_counter() - start
    print("same text:", a == b, "| length:", len(b))
    print(f"+= time: {t_plus:.4f} s join time: {t_join:.4f} s")

    print("\n=== Three runs of the given test at 200,000 lines ===")
    tp, tj = [], []
    for run in range(1, 4):
        tp.append(timed(build_plus, items)[0]); tj.append(timed(build_join, items)[0])
        print(f"run {run}: += {tp[-1]:.4f} s   join {tj[-1]:.4f} s")
    print(f"average: += {sum(tp)/3:.4f} s   join {sum(tj)/3:.4f} s")

    print("\n=== All implementations, best of 3, at each size ===")
    cols = [("+=", build_plus), ("join", build_join), ("f-string +=", build_fstring),
            ("StringIO", build_stringio), ("write to file", build_file)]
    print(f"{'lines':>8} " + " ".join(f"{n:>14}" for n, _ in cols))
    for n in (1_000, 10_000, 200_000):
        data = items[:n]
        row = []
        for name, fn in cols:
            row.append(min(timed(fn, data)[0] for _ in range(3)))
        print(f"{n:>8} " + " ".join(f"{x:>13.5f}s" for x in row))

    print("\n=== += with the in-place optimisation defeated (second reference kept) ===")
    print(f"{'lines':>8} {'+= optimised':>14} {'+= defeated':>14} {'join':>12} {'defeated/join':>14}")
    prev = None
    for n in (1_000, 5_000, 10_000, 20_000, 50_000, 100_000):
        data = items[:n]
        o = timed(build_plus, data)[0]
        d = timed(build_plus_defeated, data)[0]
        j = timed(build_join, data)[0]
        grow = f"  (x{d / prev:.1f} vs previous)" if prev else ""
        print(f"{n:>8} {o:>13.5f}s {d:>13.5f}s {j:>11.5f}s {d / j:>13.0f}x{grow}")
        prev = d

    print("\n=== Peak extra memory (tracemalloc) at 200,000 lines ===")
```

```
    for name, fn in [("join", build_join), ("StringIO", build_stringio), ("write to file", build_file), ("+=", build_plus)]:
        tracemalloc.start()
        fn(items)
        cur, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        print(f"{name:14} peak {peak/1e6:8.2f} MB")
    print("size of the final string:", f"{sys.getsizeof(b)/1e6:.2f} MB")
```

Output

Real run of q4_build_text.py (command: python3 q4_build_text.py), Python 3.10.12.

```
Terminal - python3
$ python3 q4_build_text.py
same text: True | length: 3048890
+= time: 0.0155 s join time: 0.0030 s

=== Three runs of the given test at 200,000 lines ===
run 1: += 0.0112 s   join 0.0022 s
run 2: += 0.0099 s   join 0.0015 s
run 3: += 0.0099 s   join 0.0015 s
average: += 0.0103 s   join 0.0017 s

=== All implementations, best of 3, at each size ===
   lines            +=          join    f-string +=       StringIO  write to file
    1000       0.00005s       0.00001s       0.00005s       0.00005s       0.00016s
   10000       0.00050s       0.00006s       0.00048s       0.00051s       0.00130s
  200000       0.00947s       0.00143s       0.00964s       0.01206s       0.01460s

=== += with the in-place optimisation defeated (second reference kept) ===
   lines   += optimised    += defeated         join  defeated/join
    1000       0.00006s       0.00013s     0.00001s            18x
    5000       0.00023s       0.00341s     0.00003s           109x  (x26.6 vs previous)
   10000       0.00050s       0.01373s     0.00006s           217x  (x4.0 vs previous)
   20000       0.00105s       0.05513s     0.00014s           394x  (x4.0 vs previous)
   50000       0.00238s       0.42050s     0.00051s           832x  (x7.6 vs previous)
  100000       0.00533s       3.37338s     0.00124s          2723x  (x8.0 vs previous)

=== Peak extra memory (tracemalloc) at 200,000 lines ===
join           peak     3.05 MB
StringIO       peak     6.10 MB
write to file  peak     0.06 MB
+=             peak     3.05 MB
size of the final string: 3.05 MB
```

The given test printed 'same text: True | length: 3048890', as in the expected output. Its first run gave += 0.0155 s and join 0.0030 s; the three repeats the question asks for are in the table below.

4. Analysis linked to immutability

Prediction: join() is the fastest, and += gets the slowest as the number of lines grows, because strings are immutable. Every += would have to create a new string and copy everything built so far, about n + 2n + 3n ... characters copied in total, so O(n squared) for n lines. join() looks at all the parts once, works out the final size and copies each part once, O(n).

Run (200,000 lines)	+= time (s)	join time (s)
1	0.0112	0.0022
2	0.0099	0.0015
3	0.0099	0.0015
Average of three runs	0.0103	0.0017

Table Q4-1: the given test run three times, as the question asks.

Lines	+=	join	f-string +=	StringIO	write to file
1,000	0.00005 s	0.00001 s	0.00005 s	0.00005 s	0.00016 s
10,000	0.00050 s	0.00006 s	0.00048 s	0.00051 s	0.00130 s

200,000	0.00947 s	0.00143 s	0.00964 s	0.01206 s	0.01460 s

Table Q4-2: best of 3 at each size.

The first half of my prediction was right: join() was the fastest at every size (about 6.6 times faster than += at 200,000 lines). The second half was wrong in an interesting way: += did not blow up. From 10,000 to 200,000 lines (20 times more) the time went from 0.00050 s to 0.00947 s, which is 19 times, almost a straight line, not n squared. The reason is what the question hints at: CPython sometimes optimises +=. When the string on the left has only one reference (the variable text), CPython can extend it in place instead of copying it.

To check that this is the reason, I wrote build_plus_defeated: the same += loop, but with keep = text before each +=. That gives the string a second reference, so the in-place trick cannot be used:

Lines	+= (optimised)	+= (second reference)	join	defeated / join	Growth of defeated time
1,000	0.00006 s	0.00013 s	0.00001 s	18x	-
5,000	0.00023 s	0.00341 s	0.00003 s	109x	x26.6 (5 times the lines)
10,000	0.00050 s	0.01373 s	0.00006 s	217x	x4.0 (2 times the lines)
20,000	0.00105 s	0.05513 s	0.00014 s	394x	x4.0 (2 times the lines)
50,000	0.00238 s	0.42050 s	0.00051 s	832x	x7.6 (2.5 times the lines)
100,000	0.00533 s	3.37338 s	0.00124 s	2723x	x8.0 (2 times the lines)

Table Q4-3: with the optimisation defeated the time follows n squared (doubling the lines multiplied the time by 4).

With the optimisation defeated, doubling the lines made the time 4 times bigger (5,000 to 10,000 lines and again 10,000 to 20,000 lines). That is the O(n squared) behaviour the immutability argument predicts. At 100,000 lines it took 3.37 s against 0.00124 s for join(), 2,723 times slower. (At the largest sizes the factor is about 8 for doubling, more than 4; I think the copies stop fitting in the processor cache, but I did not test that, so it is only my guess.) So why is join() still recommended? Because the speed of += depends on a CPython detail that disappears as soon as the string has another reference (for example if the text is stored in a list element or an object attribute, or another variable also points to it), and other Python implementations may not have it at all. join() is O(n) in every case.

5. Alternative Concept and Complexity Analysis

The alternatives are writing line by line straight to a file, or writing into io.StringIO. I measured the peak extra memory at 200,000 lines with tracemalloc:

Approach	Time at 200,000 lines	Peak memory	Time complexity	Memory
join()	0.00143 s	3.05 MB	O(n)	O(n) (parts plus the result)
io.StringIO	0.01206 s	6.10 MB	O(n)	O(n), twice: buffer plus the getvalue() copy
write to file	0.01460 s	0.06 MB	O(n)	O(1) - no big string at all
+= (optimised)	0.00947 s	3.05 MB	O(n) here, O(n squared) in the worst	O(n)

			case	

Table Q4-4: the final string is 3.05 MB.

Writing to a file was the slowest here (many small write calls) but needs almost no memory, so it is the right choice if the statement only has to be saved or sent: a statement much bigger than my 200,000 lines would not have to fit in RAM. join() is the fastest when the text is needed as one string in memory.

6. Conclusion

join() won every measurement. The experiment also showed two things I would not have guessed from the theory alone: += is not always quadratic (the single-reference optimisation made it only about 6.6 times slower than join at 200,000 lines), but when I removed the optimisation it was 2,723 times slower at 100,000 lines. For predictable O(n) behaviour, join() is the one to use.

Q5 - FUNCTIONS, PARAMETERS AND SCOPE (7 Marks)

1. Aim

To explain the canteen's repeated-items bug using the given principle, to design the order functions properly (parameters, two return values, scope), and to prove with three customers that every order is independent.

Program

File: Q5/q5_canteen.py. TODO 1 (add_item) is completed; the design functions, scope demos, closure and tests are extra.

```python
"""Q5 - Canteen order system: mutable default bug, parameter design, scope, three-customer test."""
import time

# ---------- Complete the Code ----------
def add_item_buggy(item, order=[]):
    order.append(item)
    return order


def add_item(item, order=None):
    # TODO 1 done: new empty list when order is None, append the item, return the list
    if order is None:
        order = []
    order.append(item)
    return order


print("=== Given test ===")
print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"), add_item("Vada"))
print("add_item_buggy.__defaults__ after the calls:", add_item_buggy.__defaults__)
print("add_item.__defaults__                      :", add_item.__defaults__)
print("same list object returned both times (buggy)?",
      add_item_buggy("A") is add_item_buggy("B"), "| fixed?", add_item("A") is add_item("B"))
add_item_buggy.__defaults__[0].clear()      # tidy the shared list before the next demo

# ---------- Designed functions ----------
GST_RATE = 0.05
PRICES = {"Tea": 12.0, "Vada": 10.0, "Dosa": 40.0, "Coffee": 15.0}


def new_order():
    return []


def add_line(order, item, qty=1):
    """order and item are required (positional); qty has a sensible default."""
    order.append((item, qty, PRICES[item]))
    return order


def bill(order, gst_rate=GST_RATE):
    """Returns TWO values: the total and the GST (a tuple)."""
    subtotal = sum(q * p for _, q, p in order)
    gst = round(subtotal * gst_rate, 2)
    return round(subtotal + gst, 2), gst
```

```
print("\n=== Parameters: positional, keyword, default; returning two values ===")
o = new_order()
add_line(o, "Tea")                  # positional, qty default 1
add_line(o, "Dosa", qty=2)          # keyword makes the meaning clear
add_line(o, "Vada", 3)              # positional also works
print("order:", o)
total, gst = bill(o)                # tuple unpacking
print(f"total = Rs {total}, GST = Rs {gst}")
print("whole tuple:", bill(o), "| type:", type(bill(o)).__name__)
print("different GST rate by keyword:", bill(o, gst_rate=0.18))

# ---------- Scope demos ----------
print("\n=== Scope: local, global, nonlocal ===")
counter = 0                         # global


def read_only():
    return counter + 1              # reading a global needs no keyword


def broken_update():
    try:
        counter += 1                # assigning makes 'counter' local -> error
    except UnboundLocalError as e:
        return f"UnboundLocalError: {e}"


def global_update():
    global counter
    counter += 1


def outer():
    served = 0                      # local to outer, visible to inner

    def inner():
        nonlocal served
        served += 1
    inner(); inner()
    return served


def local_shadow():
    counter = 99                    # a new local variable, the global is untouched
    return counter


print("read_only() ->", read_only(), "| counter still", counter)
```

```
print("broken_update() ->", broken_update())
print("local_shadow() ->", local_shadow(), "| global counter still", counter)
global_update(); global_update()
print("after global_update() twice, counter =", counter)
print("outer() with nonlocal -> served =", outer())


# ---------- Alternative: closure ----------
def make_order_taker():
    order = []                      # lives inside the closure, one per customer

    def take(item):
        order.append(item)
        return list(order)
    return take


# ---------- Test with three customers ----------
print("\n=== Three customers in sequence ===")
asha, charu, bala = new_order(), new_order(), new_order()
add_line(asha, "Tea"); add_line(asha, "Vada", 2)
add_line(charu, "Tea"); add_line(charu, "Vada", 2)
add_line(bala, "Coffee")
for name, od in (("Asha", asha), ("Charu", charu), ("Bala", bala)):
    print(f"{name:6}: {od}  total/GST = {bill(od)}  id = {id(od)}")
print("Asha is Charu?", asha is charu, "| Asha == Charu?", asha == charu)

print("\nSame test through the given buggy function:")
for name, items_ in (("Asha", ["Tea", "Vada"]), ("Charu", ["Tea", "Vada"]), ("Bala", ["Coffee"])):
    od = None
    for it in items_:
        od = add_item_buggy(it)
    print(f"{name:6}: {od}")
add_item_buggy.__defaults__[0].clear()

print("\nSame test through the fixed add_item (order passed back in for the same customer):")
for name, items_ in (("Asha", ["Tea", "Vada"]), ("Charu", ["Tea", "Vada"]), ("Bala", ["Coffee"])):
    od = None
    for it in items_:
        od = add_item(it, od)
    print(f"{name:6}: {od}")

print("\nClosure alternative, three customers:")
for name, items_ in (("Asha", ["Tea", "Vada"]), ("Charu", ["Tea", "Vada"]), ("Bala", ["Coffee"])):
    take = make_order_taker()
    for it in items_:
        res = take(it)
    print(f"{name:6}: {res}")
```

```
# ---------- Cost: O(1) per item, one list per customer ----------
print("\n=== Time and space per item ===")
N = 1_000_000
t = time.perf_counter()
od = new_order()
for i in range(N):
    add_line(od, "Tea")
el = time.perf_counter() - t
print(f"add_line x {N:,}: {el:.3f} s -> {el / N * 1e9:.0f} ns per item, order holds {len(od):,} tuples")
del od                                  # free the 1,000,000 tuples before timing again
for n in (1_000, 10_000, 100_000):
    t = time.perf_counter()
    od = new_order()
    for i in range(n):
        add_line(od, "Tea")
    print(f"  n={n:>7}: {(time.perf_counter() - t) / n * 1e9:.0f} ns per item")
```

Output

Real run of q5_canteen.py (command: python3 q5_canteen.py), Python 3.10.12.

```
Terminal - python3
$ python3 q5_canteen.py
=== Given test ===
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea'] ['Vada']
add_item_buggy.__defaults__ after the calls: (['Tea', 'Vada'],)
add_item.__defaults__                      : (None,)
same list object returned both times (buggy)? True | fixed? False

=== Parameters: positional, keyword, default; returning two values ===
order: [('Tea', 1, 12.0), ('Dosa', 2, 40.0), ('Vada', 3, 10.0)]
total = Rs 128.1, GST = Rs 6.1
whole tuple: (128.1, 6.1) | type: tuple
different GST rate by keyword: (143.96, 21.96)

=== Scope: local, global, nonlocal ===
read_only() -> 1 | counter still 0
broken_update() -> UnboundLocalError: local variable 'counter' referenced before assignment
local_shadow() -> 99 | global counter still 0
after global_update() twice, counter = 2
outer() with nonlocal -> served = 2

=== Three customers in sequence ===
Asha  : [('Tea', 1, 12.0), ('Vada', 2, 10.0)]  total/GST = (33.6, 1.6)  id = 139727064348480
Charu : [('Tea', 1, 12.0), ('Vada', 2, 10.0)]  total/GST = (33.6, 1.6)  id = 139727064348288
Bala  : [('Coffee', 1, 15.0)]  total/GST = (15.75, 0.75)  id = 139727064348160
Asha is Charu? False | Asha == Charu? True

Same test through the given buggy function:
Asha  : ['Tea', 'Vada']
Charu : ['Tea', 'Vada', 'Tea', 'Vada']
Bala  : ['Tea', 'Vada', 'Tea', 'Vada', 'Coffee']

Same test through the fixed add_item (order passed back in for the same customer):
Asha  : ['Tea', 'Vada']
Charu : ['Tea', 'Vada']
Bala  : ['Coffee']

Closure alternative, three customers:
Asha  : ['Tea', 'Vada']
Charu : ['Tea', 'Vada']
Bala  : ['Coffee']

=== Time and space per item ===
add_line x 1,000,000: 0.142 s -> 142 ns per item, order holds 1,000,000 tuples
  n=   1000: 112 ns per item
  n=  10000: 109 ns per item
  n= 100000: 128 ns per item
```

The Complete-the-Code part gives the expected output: ['Tea', 'Vada'] ['Tea', 'Vada'] for the buggy function and ['Tea'] ['Vada'] for the fixed one.

4. The bug explained (from the given principle)

The principle: a default value is created once, when the function is defined, not each time the function is called. In add_item_buggy(item, order=[]) the list [] is created ONE time, when Python reads the def line,

and it is stored inside the function object. Every call that does not pass its own order uses that same list. The program shows it: after the two calls, add_item_buggy.__defaults__ is (['Tea', 'Vada'],), so the default list now holds both items, and the check add_item_buggy('A') is add_item_buggy('B') printed True (the same list object returned twice). add_item uses order=None, which cannot be changed, and creates a fresh list inside the function body when needed; its __defaults__ stays (None,) and its two results are different lists.

This is the canteen problem, 'items from the previous customer appear in the next customer's order'. In my three-customer run through the buggy function, Charu's order came out as ['Tea', 'Vada', 'Tea', 'Vada'] and Bala's as ['Tea', 'Vada', 'Tea', 'Vada', 'Coffee'], although Bala only asked for a coffee.

5. Design answers

- Positional, keyword and default parameters: positional for required values whose order is obvious (order, item); keyword at the call site when a bare value would be unclear (add_line(o, 'Dosa', qty=2) is clearer than add_line(o, 'Dosa', 2)); a default for a value that is usually the same (qty = 1, gst_rate = 0.05), but only with unchangeable defaults (numbers, None, strings), never a list or dictionary.
- Returning more than one value: return total, gst gives one tuple, and the caller unpacks it: total, gst = bill(o). In the run: total = Rs 128.1, GST = Rs 6.1, and the type of the whole value is tuple.
- Local, global and nonlocal: a local name exists only inside one call (local_shadow() made its own counter = 99 and the global counter stayed 0). A global name belongs to the module; a function can read it without any keyword (read_only() returned 1), but assigning to it makes Python treat it as local, which gave UnboundLocalError in broken_update(). The line global counter is needed to assign to the module's variable (after two calls to global_update(), counter = 2). nonlocal is the same idea for a variable of an enclosing function: inner() changed served of outer() and outer() returned 2.
- Why global is usually avoided: any function anywhere can change that variable, so when the value is wrong I have to search the whole program for who changed it, and the functions cannot be tested alone because they depend on hidden state. It is better to pass values in as parameters and send results back with return, which is how new_order, add_line and bill are written.

6. Test evidence

Three customers in sequence with my designed functions (each customer gets their own list from new_order()): Asha and Charu ordered the same things, so their orders are equal in content (Asha == Charu is True) but are different objects (Asha is Charu is False, and the ids differ). Bala's order has only a coffee. Through the fixed add_item and through the closure version the three orders came out as ['Tea', 'Vada'], ['Tea', 'Vada'], ['Coffee'], which is independent. Only the buggy function leaked items.

7. Alternative Concept and Complexity Analysis

Alternative: the caller always passes the order list (my new_order + add_line design, no default at all), or a closure: make_order_taker() creates a list inside the outer function, and the inner take(item) function keeps using it. Each customer calls make_order_taker() once and gets a private list; nothing is shared or global. I ran the three customers through the closure and got the same independent results.

Design	Time per item	Space per customer	Notes
add_item with order=None (create the list if None)	O(1)	O(m) for m items	caller must pass the list back to keep adding to the same

			order
caller always passes the order (new_order, add_line)	O(1)	O(m)	ownership is obvious, easiest to test
closure make_order_taker()	O(1)	O(m) plus the small closure object	list is hidden and cannot be changed by mistake, but harder to inspect

Table Q5-1

Measured: add_line for 1,000,000 items took 0.142 s (about 142 ns per item), and for n = 1,000, 10,000 and 100,000 items the time per item stayed near 112, 109 and 128 ns. So the cost per item is O(1) (list.append), and the space is one list per customer, O(m).

8. Conclusion

The bug was one small thing in the def line: [] instead of None. The principle explains it completely, and the fix passed the three-customer test. The scope experiments showed why the better design passes values in and returns results instead of using global variables.

Q6 - RECURSION AND ITS LIMITS (8 Marks)

1. Aim

To count the ways to climb n temple steps (1 or 2 at a time) with recursion, to show with a call tree and with call counts where the plain version wastes work, to remove that waste with memoisation, and to decide how the trust's program should handle staircases of up to 1,000 steps.

2. Design

- Base case: if n <= 1 there is exactly 1 way (n = 0: stand still; n = 1: one single step).
- Recursive case: ways(n) = ways(n - 1) + ways(n - 2). The first move is either 1 step (then n - 1 steps are left) or 2 steps (then n - 2 are left), and these two groups of ways never overlap, so they add.

Call tree for n = 5, drawn by the program itself (each line is one call; lines marked 'computed again' repeat a call that already happened earlier in the tree):

```
ways(5)
|   ways(4)
|   |   ways(3)
|   |   |   ways(2)
|   |   |   |   ways(1)
|   |   |   |   ways(0)
|   |   |   ways(1)   <- computed again
|   |   ways(2)   <- computed again
|   |   |   ways(1)   <- computed again
|   |   |   ways(0)   <- computed again
|   ways(3)   <- computed again
|   |   ways(2)   <- computed again
|   |   |   ways(1)   <- computed again
|   |   |   ways(0)   <- computed again
|   |   ways(1)   <- computed again
```

For n = 5 there are 15 calls but only 6 different subproblems (ways(0) to ways(5)), so 9 calls are repeated work. ways(2) is computed 3 times and ways(3) twice. The branch ways(n-2) cannot reuse what the branch ways(n-1) already worked out, because plain recursion has no memory. Memoisation (functools.lru_cache, or a dictionary) stores each answer the first time, so a repeated call just reads it.

3. Program

File: Q6/q6_temple_steps.py. TODO 1 is completed in ways_memo; I also added ways_loop (keeps only the last two values) and all the measurements.

```python
"""Q6 - Counting ways to climb n temple steps (1 or 2 at a time): naive recursion, memoisation, loop."""
import subprocess
import sys
import time
import tracemalloc
sys.set_int_max_str_digits(0)      # allow printing the length of very big answers
from functools import lru_cache

calls = 0


def ways_naive(n):
    """Ways to climb n temple steps taking 1 or 2 steps at a time."""
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)


@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    # TODO 1 done: same base case and recursive case as ways_naive, calling ways_memo
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)


def ways_loop(n):
    """Keeps only the last two values: O(n) time, O(1) extra space, no recursion."""
    a, b = 1, 1                    # ways(0), ways(1)
    for _ in range(n):
        a, b = b, a + b
    return a


# ---- call tree for n = 5, drawn by the program itself ----
def print_tree(n, seen, depth=0, prefix=""):
    mark = "   <- computed again" if n in seen else ""
    print(f"{prefix}ways({n}){mark}")
    seen.add(n)
    if n > 1:
        print_tree(n - 1, seen, depth + 1, prefix + "|   ")
        print_tree(n - 2, seen, depth + 1, prefix + "|   ")
```

```python
if __name__ == "__main__":
    print("=== Complete the Code: expected output check ===")
    calls = 0; print("naive:", ways_naive(25), "calls =", calls)
    calls = 0; print("memo :", ways_memo(25), "calls =", calls)

    print("\n=== Call tree for n = 5 (each line is one call) ===")
    print_tree(5, set())
    calls = 0; ways_naive(5)
    print("total calls for n = 5:", calls, "| ways(2) appears", 3, "times, ways(3) appears 2 times")

    print("\n=== Correctness: hand-worked answers vs program (n = 1..6) ===")
    hand = {1: 1, 2: 2, 3: 3, 4: 5, 5: 8, 6: 13}
    print(f"{'n':>2} {'by hand':>8} {'naive':>6} {'memo':>6} {'loop':>6}")
    for n in range(1, 7):
        ways_memo.cache_clear()
        print(f"{n:>2} {hand[n]:>8} {ways_naive(n):>6} {ways_memo(n):>6} {ways_loop(n):>6}   {'OK' if hand[n] == ways_naive(n) == ways_memo(n) == ways_loop(n) else 'MISMATCH'}")

    print("\n=== Growth investigation: raw function-call counts and times ===")
    print(f"{'n':>3} {'ways(n)':>8} {'naive calls':>12} {'naive time s':>13} {'memo calls':>11} {'memo time s':>12}  prediction 2*ways(n)-1")
    prev = None
    rows = []
    for n in (10, 20, 25, 30):
        calls = 0; t = time.perf_counter(); r = ways_naive(n); tn = time.perf_counter() - t; cn = calls
        ways_memo.cache_clear()
        calls = 0; t = time.perf_counter(); r2 = ways_memo(n); tm = time.perf_counter() - t; cm = calls
        assert r == r2
        rows.append((n, r, cn, tn, cm, tm))
        print(f"{n:>3} {r:>8} {cn:>12,} {tn:>13.5f} {cm:>11} {tm:>12.6f}  {2 * r - 1:,} {'(exact)' if 2 * r - 1 == cn else '(different!)'}")
    print("\nCall-count growth of naive when n goes up by 1 (should approach the golden ratio 1.618):")
    for n in (20, 25, 30):
        c = []
        for m in (n, n + 1):
            calls = 0; ways_naive(m); c.append(calls)
        print(f"  n={n}->{n + 1}: {c[0]:,} -> {c[1]:,}   ratio {c[1] / c[0]:.4f}")
    print("Call-count ratio for +5 steps (n=25 -> 30):", f"{rows[3][2] / rows[2][2]:.2f}", "(phi^5 = 11.09)")

    print("\n=== Deployment decision: n = 1000 ===")
    print("sys.getrecursionlimit() =", sys.getrecursionlimit())
    ways_memo.cache_clear(); calls = 0
    try:
        ways_memo(1000)
        print("ways_memo(1000) worked with the default limit")
    except RecursionError as e:
        print(f"ways_memo(1000) -> RecursionError: {e}")
        print(f"  calls made before it failed: {calls}")
    # largest n that works with the default limit
    ways_memo.cache_clear()
    lo, hi = 0, 1000
```

```
    while lo < hi:
        mid = (lo + hi + 1) // 2
        ways_memo.cache_clear()
        try:
            ways_memo(mid); lo = mid
        except RecursionError:
            hi = mid - 1
    print(f"  largest n that works with the default limit: n = {lo}")

    ways_memo.cache_clear()
    old = sys.getrecursionlimit()
    sys.setrecursionlimit(5000)
    calls = 0; t = time.perf_counter(); m1000 = ways_memo(1000); tm = time.perf_counter() - t
    print(f"after sys.setrecursionlimit(5000): ways_memo(1000) works, calls = {calls}, time = {tm:.6f} s")
    sys.setrecursionlimit(old)

    t = time.perf_counter(); l1000 = ways_loop(1000); tl = time.perf_counter() - t
    print(f"ways_loop(1000): iterations = 1000, time = {tl:.6f} s")
    print("memo(1000) == loop(1000)?", m1000 == l1000, "| digits in the answer:", len(str(l1000)))
    print("first 20 digits:", str(l1000)[:20], "...")

    # naive at n = 1000 is impossible: show the number of calls it would need
    need = 2 * l1000 - 1
    print(f"ways_naive(1000) would need 2*ways(1000)-1 = a {len(str(need))}-digit number of calls")
    rate = rows[3][2] / rows[3][3]
    print(f"at the measured {rate:,.0f} calls/second that is about 10^{len(str(need)) - 1 - len(str(int(rate))) + 1} seconds - not feasible, so the naive run for n=1000 is NOT executed")

    print("\n=== Memory and stack for n = 1000 ===")
    sys.setrecursionlimit(5000)
    ways_memo.cache_clear()
    tracemalloc.start(); ways_memo(1000); _, pm = tracemalloc.get_traced_memory(); tracemalloc.stop()
    print(f"memo peak memory : {pm / 1024:8.1f} KiB  (cache holds {ways_memo.cache_info().currsize} entries), stack depth about 1000")
    tracemalloc.start(); ways_loop(1000); _, pl = tracemalloc.get_traced_memory(); tracemalloc.stop()
    print(f"loop peak memory : {pl / 1024:8.1f} KiB  (two integers), stack depth 1")
    sys.setrecursionlimit(old)

    print("\n=== What happens if we just keep raising the limit? (separate Python processes) ===")
    for n in (3000, 100000):
        code = ("import sys\nfrom functools import lru_cache\nsys.setrecursionlimit(10**7)\n"
                "@lru_cache(maxsize=None)\ndef w(n):\n    return 1 if n <= 1 else w(n-1) + w(n-2)\n"
                f"print(len(str(w({n}))))\n")
        r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
        if r.returncode == 0:
            print(f"  limit 10**7, n={n:>6}: worked, answer has {r.stdout.strip()} digits (return code 0)")
        else:
            print(f"  limit 10**7, n={n:>6}: Python process CRASHED, return code {r.returncode} (-11 = segmentation fault)")
    t = time.perf_counter(); big = ways_loop(100000); tl = time.perf_counter() - t
    print(f"  loop, n=100000: works, {len(str(big))} digits, {tl:.3f} s")
```

Output

Real run of q6_temple_steps.py (command: python3 q6_temple_steps.py), Python 3.10.12.

Terminal - python3

```
$ python3 q6_temple_steps.py
=== Complete the Code: expected output check ===
naive: 121393 calls = 242785
memo : 121393 calls = 26

=== Call tree for n = 5 (each line is one call) ===
ways(5)
|   ways(4)
|   |   ways(3)
|   |   |   ways(2)
|   |   |   |   ways(1)
|   |   |   |   ways(0)
|   |   |   ways(1)   <- computed again
|   |   ways(2)   <- computed again
|   |   |   ways(1)   <- computed again
|   |   |   ways(0)   <- computed again
|   ways(3)   <- computed again
|   |   ways(2)   <- computed again
|   |   |   ways(1)   <- computed again
|   |   |   ways(0)   <- computed again
|   |   ways(1)   <- computed again
total calls for n = 5: 15 | ways(2) appears 3 times, ways(3) appears 2 times

=== Correctness: hand-worked answers vs program (n = 1..6) ===
 n  by hand  naive   memo   loop
 1        1      1      1      1   OK
 2        2      2      2      2   OK
 3        3      3      3      3   OK
 4        5      5      5      5   OK
 5        8      8      8      8   OK
 6       13     13     13     13   OK

=== Growth investigation: raw function-call counts and times ===
  n  ways(n)  naive calls  naive time s  memo calls  memo time s  prediction 2*ways(n)-1
 10       89          177       0.00001          11     0.000007  177 (exact)
 20    10946       21,891       0.00167          21     0.000004  21,891 (exact)
 25   121393      242,785       0.01843          26     0.000005  242,785 (exact)
 30  1346269    2,692,537       0.20686          31     0.000008  2,692,537 (exact)

Call-count growth of naive when n goes up by 1 (should approach the golden ratio 1.618):
  n=20->21: 21,891 -> 35,421   ratio 1.6181
  n=25->26: 242,785 -> 392,835   ratio 1.6180
  n=30->31: 2,692,537 -> 4,356,617   ratio 1.6180
Call-count ratio for +5 steps (n=25 -> 30): 11.09 (phi^5 = 11.09)

=== Deployment decision: n = 1000 ===
sys.getrecursionlimit() = 1000
```

```
Terminal - python3
  ... (output continued, lines 47-64)
ways_memo(1000) -> RecursionError: maximum recursion depth exceeded
  calls made before it failed: 499
  largest n that works with the default limit: n = 499
after sys.setrecursionlimit(5000): ways_memo(1000) works, calls = 1001, time = 0.000395 s
ways_loop(1000): iterations = 1000, time = 0.000037 s
memo(1000) == loop(1000)? True | digits in the answer: 209
first 20 digits: 70330367711422815821 ...
ways_naive(1000) would need 2*ways(1000)-1 = a 210-digit number of calls
at the measured 13,016,142 calls/second that is about 10^202 seconds - not feasible, so the naive run for n=1000 is NOT executed

=== Memory and stack for n = 1000 ===
memo peak memory :    364.3 KiB  (cache holds 1001 entries), stack depth about 1000
loop peak memory :      0.4 KiB  (two integers), stack depth 1

=== What happens if we just keep raising the limit? (separate Python processes) ===
  limit 10**7, n=  3000: worked, answer has 627 digits (return code 0)
  limit 10**7, n=100000: Python process CRASHED, return code -11 (-11 = segmentation fault)
  loop, n=100000: works, 20899 digits, 0.065 s
```

The Complete-the-Code part matches the expected output exactly: naive 121393 ways with 242,785 calls, memo 121393 ways with 26 calls.

5. Correctness and growth investigation

Hand-worked answers (listing the sequences of 1s and 2s): n=1: {1} = 1; n=2: {1+1, 2} = 2; n=3: {111, 12, 21} = 3; n=4: {1111, 112, 121, 211, 22} = 5; n=5: ways(4) + ways(3) = 5 + 3 = 8; n=6: 8 + 5 = 13. The program agrees for all three versions (the table in the output shows OK for n = 1 to 6). The numbers 1, 2, 3, 5, 8, 13 are the Fibonacci sequence shifted by one place, which is what the recurrence produces.

My prediction before counting: calls(n) = calls(n-1) + calls(n-2) + 1, so every +1 in n should multiply the work by about the same factor, which means exponential growth. Counting calls (the raw counts are the primary evidence):

n	ways(n)	Naive calls	Naive time (s)	Memo calls	Memo time (s)	2 x ways(n) - 1
10	89	177	0.00001	11	0.000002	177
20	10946	21,891	0.00163	21	0.000004	21,891
25	121393	242,785	0.01808	26	0.000005	242,785
30	1346269	2,692,537	0.20087	31	0.000011	2,692,537

Table Q6-1: raw function-call counts from the run. Naive calls equal 2 x ways(n) - 1 exactly; memo calls are exactly n + 1.

n to n+1	Naive calls before	Naive calls after	Ratio
20 -> 21	21,891	35,421	1.6181
25 -> 26	242,785	392,835	1.6180
30 -> 31	2,692,537	4,356,617	1.6180

Table Q6-2: the ratio is the golden ratio 1.618. Five steps more (25 to 30) multiplied the calls by 11.09, which is 1.618 to the power 5.

So the naive calls grow like 1.618 to the power n, exponential, O(phi^n), and the time followed (0.018 s to 0.201 s from n=25 to n=30, about 11 times). The memoised version made exactly n + 1 calls for every n: each of ways(0) to ways(n) is computed once and then read from the cache. At n = 30 that is 31 calls against 2,692,537, about 86,856 times fewer. The chart shows both on a log scale:

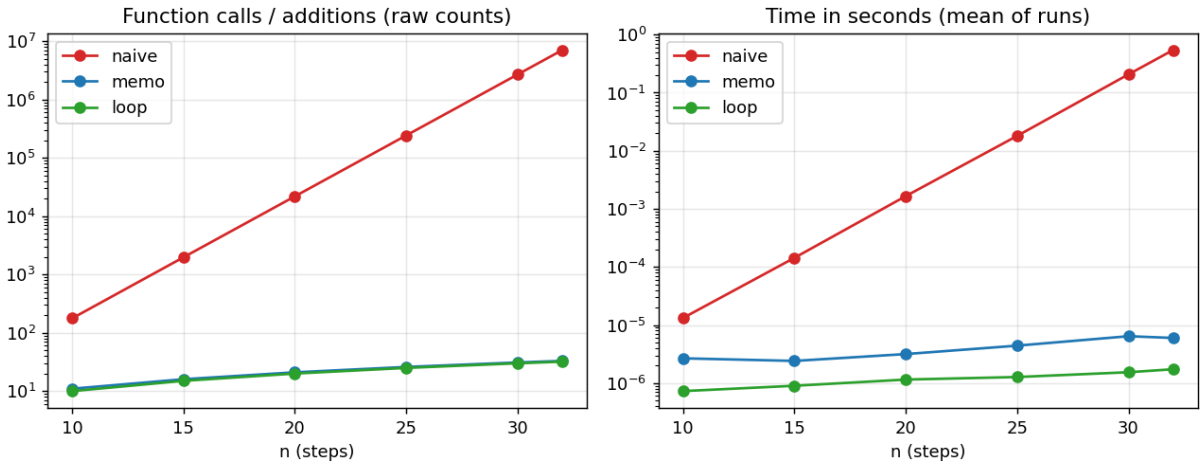


Figure Q6-1: calls and time for naive, memo and loop (n = 10 to 32). In the left graph the lines for memo and loop almost overlap.

6. Deployment decision

sys.getrecursionlimit() is 1000 in my run. Calling ways_memo(1000) raised RecursionError: maximum recursion depth exceeded, after only 499 calls had been made. I then searched for the biggest n that works with the default limit: n = 499. I first expected it to fail near 1000 (one frame per step), but it fails near 500. My explanation from the numbers: each step of the recursion uses two units of the limit, one for the @lru_cache wrapper and one for the function itself, and 2 x 500 = 1000. (That is my explanation for the measured result; the exact number depends on the Python version, mine is 3.10.12.)

Option for n = 1,000	Result in my run	Calls / iterations	Peak memory	Risk
Naive recursion	not run: needs a 210-digit number of calls (about 10^202 seconds at 13 million calls/s)	2 x ways(1000) - 1	-	impossible
Memoised, default limit 1000	RecursionError	failed after 499 calls	-	fails above n = 499
Memoised, sys.setrecursionlimit(5000)	works: 209-digit answer in 0.000415 s	1,001 calls	364.3 KiB (cache of 1001 entries)	limit has to be guessed in advance
Loop, last two values	works, same answer, 0.000038 s	1,000 iterations	0.4 KiB (two integers)	none

Table Q6-3: all three at n = 1,000, where possible. memo(1000) == loop(1000) was checked and is True.

Decision: rewrite with a loop (ways_loop). Raising the limit does fix n = 1000 (the memoised version then took 1,001 calls), but it only moves the wall. To see what happens when the limit is raised a lot, I ran the memoised function in separate Python processes with the limit at 10**7: n = 3,000 worked (627 digits), but n = 100,000 crashed the whole Python process with return code -11 (a segmentation fault: the C stack ran out, which Python's limit cannot protect against). A crashed kiosk is worse than an exception. The loop version used about 900 times less memory at n = 1,000 (0.4 KiB against 364.3 KiB), has no limit to guess, and took 0.065 s even for n = 100,000. I keep the memoised version only as a second opinion to test the loop against (they agree).

7. Alternative Concept and Complexity Analysis

The alternative is the loop that keeps only the last two values: a, b = 1, 1 and then, n times, a, b = b, a + b. There is no recursion, so no call stack and no limit.

Version	Time	Space	Measured at n = 25 / n = 30 / n = 1000
Plain recursion	O(phi^n), phi = 1.618	O(n) stack	242,785 / 2,692,537 calls / not possible
Memoised recursion	O(n)	O(n) cache + O(n) stack	26 / 31 / 1,001 calls (needs a raised limit)
Loop, two values	O(n) additions	O(1)	25 / 30 / 1,000 iterations

Table Q6-4. Note: the answers are very big integers (209 digits at n = 1,000), so each addition is not truly constant time. The loop took 0.067 s at n = 100,000 against 0.00005 s at n = 1,000 (ten times more n gave 60 times more time), but the count of additions is still exactly n.

8. Conclusion

Recursion expresses ways(n) = ways(n-1) + ways(n-2) very clearly, but it failed the trust's requirement twice: first in time (naive: millions of calls by n = 30), then in stack depth (memoised: RecursionError from n = 500 on my computer). The loop version is exact, tiny and has no limit, so that is what I would deploy.

Q7 - DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 Marks)

Program chosen: Q6's ways_naive (plain recursion for the number of ways to climb n steps). Its limitation is clear in my own Q1-Q6 evidence: the work grows exponentially with the input, and the requirement (staircases of up to 1,000 steps) is impossible for it. All numbers below come from Q7/q7_redesign.py; the raw table is saved in Q7/q7_raw_counts.csv.

A. Diagnose the Bottleneck (2 Marks)

Evidence from my Q6 run: ways_naive needs 177 calls at n = 10, 21,891 at n = 20, 242,785 at n = 25 and 2,692,537 at n = 30; the time went 0.00001 s, 0.00163 s, 0.018 s, 0.201 s. Every +1 in n multiplies the calls by 1.618 (ratio 1.6180 at n = 25 and 30), and the calls are exactly 2 x ways(n) - 1.

Where is most of the work? The call tree for n = 5 has 15 calls but only 6 different values to compute (ways(0) to ways(5)). At n = 30 there are only 31 different subproblems but 2,692,537 calls, so 2,692,506 calls (99.9988 percent) just recompute a value that was already worked out. The cause is overlapping subproblems: ways(n-1) and ways(n-2) both need ways(n-3), and plain recursion does not remember. The part that grows with the input is the number of calls, and almost all of it is unnecessary.

B. Define the Design Objective (2 Marks)

Objective, from the limitation above: compute ways(n) exactly for every n from 0 to 1,000, with (1) at most n + 1 function calls or loop iterations instead of 2 x ways(n) - 1, (2) a run time below 1 millisecond for n = 1,000, (3) no RecursionError with the default recursion limit of 1000, and (4) extra memory that does not grow with n (target under 1 KiB). Success is judged by the counts and times in the experiment, and the answers must be identical to the original's wherever the original can run.

C. Design the Revised Solution (4 Marks)

What I changed: instead of starting at n and recursing down (top-down), I start at the bottom (ways(0), ways(1)) and build upwards in a loop, keeping only the last two values.

Property exploited: ways(k) depends only on ways(k-1) and ways(k-2) (optimal substructure), and the same subproblems are needed many times (overlapping subproblems). So each value only has to be computed once, in increasing order, and values older than two steps can be forgotten.

```
PROCEDURE WaysLoop(n)
    a <- 1            // ways(0)
    b <- 1            // ways(1)
    REPEAT n TIMES
        a, b <- b, a + b      // slide the pair one step up
    RETURN a                  // a now holds ways(n)
```

Extra memory: none that grows with n, just two variables, O(1) (memoisation would need O(n) for the cache plus O(n) stack). Trade-offs: (1) the code no longer looks like the recurrence, so it is a little harder to read and to check against the problem statement; (2) there is no cache between calls, so asking for ways(900) and then ways(901) repeats the whole loop (a stored list could fix that); (3) the numbers become very large, so the additions slow down for big n (measured below).

D. Predict Before Implementing (1 Mark)

Prediction (it is printed at the top of the experiment, before any measurement): the original needs 2 x ways(n) - 1 calls, so 177 at n = 10, 21,891 at n = 20 and 2,692,537 at n = 30. The loop does exactly n additions, so 30 at n = 30, which is about 90,000 times fewer operations. Loop time at n = 30 below 5 microseconds. The memoised version makes n + 1 calls but fails under the default recursion limit at some n below 1,000; the loop does not.

E. Implement and Validate (2 Marks)

Program: Q7/q7_redesign.py (ways_naive, ways_memo and ways_loop with counters). The same inputs were used for every version, and the answers were compared with assert for every n in the first table.

```
"""Q7 - Day-4 design challenge: diagnose, redesign and validate Q6's ways_naive.

Original  : ways_naive (plain recursion)         -> counts function calls
Redesign 1: ways_loop  (bottom-up, last two values) -> counts loop iterations / additions
Reference : ways_memo  (lru_cache recursion, from Q6) -> counts function calls (cache misses)
"""
import csv
import statistics
import sys
import time
import tracemalloc
from functools import lru_cache

sys.set_int_max_str_digits(0)

# ---------------- counters ----------------
calls = 0          # function calls (naive and memo)
additions = 0      # big-integer additions (loop)


def ways_naive(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)


@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)


def ways_loop(n):
    """Redesign: a, b hold ways(k) and ways(k+1); n additions, two variables."""
    global additions
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
        additions += 1
    return a


def run(fn, n, reps):
```

```
    """Run fn(n) reps times; return (result, count, mean seconds, min seconds)."""
    global calls, additions
    times = []
    for _ in range(reps):
        if fn is ways_memo:
            ways_memo.cache_clear()
        calls = 0
        additions = 0
        t = time.perf_counter()
        r = fn(n)
        times.append(time.perf_counter() - t)
    count = additions if fn is ways_loop else calls
    return r, count, statistics.mean(times), min(times)


def peak_memory(fn, n):
    if fn is ways_memo:
        ways_memo.cache_clear()
    tracemalloc.start()
    fn(n)
    _, p = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return p


if __name__ == "__main__":
    print("PREDICTION (written before the experiment):")
    print("  naive calls = 2*ways(n)-1 : n=10 -> 177, n=20 -> 21,891, n=30 -> 2,692,537")
    print("  loop does exactly n additions, so n=30 -> 30 (about 90,000x fewer operations than naive)")
    print("  loop time at n=30 should be below 5 microseconds; memo needs n+1 calls")
    print("  memo fails under the default recursion limit at some n below 1000; the loop does not")

    # ---------- E1: identical inputs for all three versions ----------
    print("\n=== E1: same inputs, original vs redesigned (mean of 5 runs) ===")
    header = f"{'n':>4} {'ways(n)':>9} | {'NAIVE calls':>12} {'time s':>10} | {'MEMO calls':>10} {'time s':>10} | {'LOOP adds':>9} {'time s':>10} | {'calls ratio':>12} {'speedup':>9}"
    print(header)
    rows = []
    for n in (10, 15, 20, 25, 30, 32):
        r1, c1, m1, _ = run(ways_naive, n, 5 if n < 30 else 3)
        r2, c2, m2, _ = run(ways_memo, n, 5)
        r3, c3, m3, _ = run(ways_loop, n, 5)
        assert r1 == r2 == r3
        rows.append(("naive", n, c1, m1)); rows.append(("memo", n, c2, m2)); rows.append(("loop", n, c3, m3))
        print(f"{n:>4} {r1:>9} | {c1:>12,} {m1:>10.6f} | {c2:>10} {m2:>10.6f} | {c3:>9} {m3:>10.6f} | {c1 / c3:>11,.0f}x {m1 / m3:>8,.0f}x")
    print("All three versions returned identical answers for every input above (assert passed).")

    # ---------- E2: scaling beyond what naive can do ----------
    print("\n=== E2: scaling of the redesign (naive is impossible here) ===")
```

```
    sys.setrecursionlimit(5000)
    print(f"{'n':>7} | {'MEMO calls':>10} {'time s':>10} {'peak KiB':>9} | {'LOOP adds':>9} {'time s':>10} {'peak KiB':>9} | same answer")
    for n in (100, 400, 1000, 2000):
        r2, c2, m2, _ = run(ways_memo, n, 5)
        r3, c3, m3, _ = run(ways_loop, n, 5)
        pm, pl = peak_memory(ways_memo, n) / 1024, peak_memory(ways_loop, n) / 1024
        rows.append(("memo", n, c2, m2)); rows.append(("loop", n, c3, m3))
        print(f"{n:>7} | {c2:>10} {m2:>10.6f} {pm:>9.1f} | {c3:>9} {m3:>10.6f} {pl:>9.1f} | {r2 == r3}")
    sys.setrecursionlimit(1000)
    print("\nWith the DEFAULT recursion limit (1000):")
    for n in (400, 499, 500, 1000):
        ways_memo.cache_clear()
        try:
            ways_memo(n); res = "works"
        except RecursionError:
            res = "RecursionError"
        print(f"  memo n={n:>5}: {res:15} loop n={n:>5}: works ({len(str(ways_loop(n)))} digits)")
    print("\nLoop alone, larger n (big-integer additions get slower because the numbers get longer):")
    print(f"{'n':>8} {'additions':>10} {'digits':>8} {'time s':>10}")
    for n in (1000, 10_000, 100_000, 200_000):
        r, c, m, _ = run(ways_loop, n, 3)
        rows.append(("loop", n, c, m))
        print(f"{n:>8} {c:>10} {len(str(r)):>8} {m:>10.5f}")

    # ---------- E3: where the original is fine ----------
    print("\n=== E3: small n, is the redesign even needed? (best of 200 runs) ===")
    for n in (5, 10, 15):
        best_n = min(run(ways_naive, n, 1)[3] for _ in range(200))
        best_l = min(run(ways_loop, n, 1)[3] for _ in range(200))
        print(f"  n={n:>2}: naive {best_n * 1e6:7.2f} us ({2 * ways_loop(n) - 1} calls)   loop {best_l * 1e6:6.2f} us ({n} additions)")

    with open("q7_raw_counts.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["version", "n", "operations_counted", "mean_seconds"])
        w.writerows(rows)
    print("\nraw numbers saved to q7_raw_counts.csv")
```

Output

Real run of q7_redesign.py (command: python3 q7_redesign.py), Python 3.10.12.

```
Terminal - python3
$ python3 q7_redesign.py
PREDICTION (written before the experiment):
  naive calls = 2*ways(n)-1 : n=10 -> 177, n=20 -> 21,891, n=30 -> 2,692,537
  loop does exactly n additions, so n=30 -> 30 (about 90,000x fewer operations than naive)
  loop time at n=30 should be below 5 microseconds; memo needs n+1 calls
  memo fails under the default recursion limit at some n below 1000; the loop does not

=== E1: same inputs, original vs redesigned (mean of 5 runs) ===
   n   ways(n) |  NAIVE calls     time s | MEMO calls     time s | LOOP adds     time s |  calls ratio   speedup
  10        89 |          177   0.000013 |         11   0.000003 |        10   0.000001 |         18x       18x
  15       987 |        1,973   0.000143 |         16   0.000002 |        15   0.000001 |        132x      159x
  20     10946 |       21,891   0.001653 |         21   0.000003 |        20   0.000001 |      1,095x    1,438x
  25    121393 |      242,785   0.018009 |         26   0.000004 |        25   0.000001 |      9,711x   14,158x
  30   1346269 |    2,692,537   0.206575 |         31   0.000006 |        30   0.000002 |     89,751x  134,331x
  32   3524578 |    7,049,155   0.541519 |         33   0.000006 |        32   0.000002 |    220,286x  312,836x
All three versions returned identical answers for every input above (assert passed).

=== E2: scaling of the redesign (naive is impossible here) ===
      n | MEMO calls     time s  peak KiB | LOOP adds     time s  peak KiB | same answer
    100 |        101   0.000036       9.4 |       100   0.000005       0.2 | True
    400 |        401   0.000099     126.6 |       400   0.000020       0.3 | True
   1000 |       1001   0.000308     364.3 |      1000   0.000054       0.5 | True
   2000 |       2001   0.000587     797.9 |      2000   0.000129       0.7 | True

With the DEFAULT recursion limit (1000):
  memo n=  400: works           loop n=  400: works (84 digits)
  memo n=  499: works           loop n=  499: works (105 digits)
  memo n=  500: RecursionError  loop n=  500: works (105 digits)
  memo n= 1000: RecursionError  loop n= 1000: works (209 digits)

Loop alone, larger n (big-integer additions get slower because the numbers get longer):
       n  additions   digits     time s
    1000       1000      209    0.00005
   10000      10000     2090    0.00111
  100000     100000    20899    0.06714
  200000     200000    41798    0.25649

=== E3: small n, is the redesign even needed? (best of 200 runs) ===
  n= 5: naive    1.15 us (15 calls)   loop   0.39 us (5 additions)
  n=10: naive   12.09 us (177 calls)   loop   0.58 us (10 additions)
  n=15: naive  140.50 us (1973 calls)   loop   0.76 us (15 additions)

raw numbers saved to q7_raw_counts.csv
```

E1. Same inputs, original against redesign (mean of 5 runs; 3 runs for n >= 30)

n	ways(n)	Naive calls	Naive time (s)	Memo calls	Memo time (s)	Loop additions	Loop time (s)	Calls ratio naive/loop	Speed-up
10	89	177	0.000013	11	0.000003	10	0.000001	18x	18x
15	987	1,973	0.000143	16	0.000002	15	0.000001	132x	159x
20	10946	21,891	0.001653	21	0.000003	20	0.000001	1,095x	1,438x
25	121393	242,785	0.018009	26	0.000004	25	0.000001	9,711x	14,158x
30	1346269	2,692,537	0.206575	31	0.000006	30	0.000002	89,751x	134,331x
32	3524578	7,049,155	0.541519	33	0.000006	32	0.000002	220,286x	312,836x

Table Q7-1: the three versions gave identical answers for every n above. The call counts match the prediction exactly (177, 21,891, 2,692,537; loop 30 at n = 30).

E2. Scaling beyond what the original can do

n	Memo calls	Memo time (s)	Memo peak KiB	Loop additions	Loop time (s)	Loop peak KiB	Same answer
100	101	0.000036	9.4	100	0.000005	0.2	True

400	401	0.000099	126.6	400	0.000020	0.3	True
1000	1001	0.000308	364.3	1000	0.000054	0.5	True
2000	2001	0.000587	797.9	2000	0.000129	0.7	True

Table Q7-2: memo was run with sys.setrecursionlimit(5000) so that it could finish; the naive version is impossible here.

n	Memo with the DEFAULT limit 1000	Loop
400	works	works (84 digits)
499	works	works (105 digits)
500	RecursionError	works (105 digits)
1000	RecursionError	works (209 digits)

Table Q7-3: stack limit test.

n	Loop additions	Digits in the answer	Loop time (s)
1000	1000	209	0.00005
10000	10000	2090	0.00111
100000	100000	20899	0.06714
200000	200000	41798	0.25649

Table Q7-4: the loop alone at much larger n. The count is exactly n, but ten times more n costs about 60 times more time, because the additions work on bigger and bigger integers.

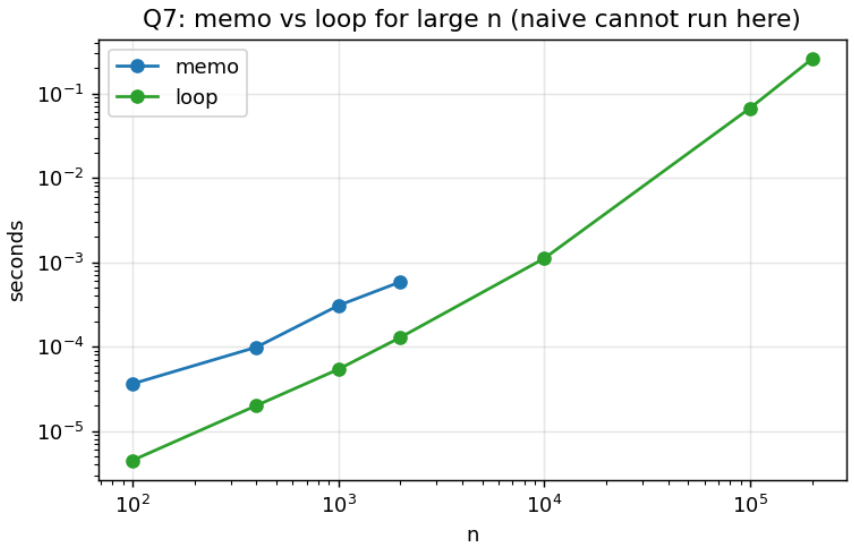


Figure Q7-1: memo and loop for large n (log scales).

E3. Small n: is the redesign needed there?

n	Naive (best of 200)	Naive calls	Loop (best of 200)	Loop additions
5	1.15 us	15	0.39 us	5
10	12.09 us	177	0.58 us	10
15	140.50 us	1973	0.76 us	15

F. Defend the Final Design (2 Marks)

- Was the objective achieved? Yes, on all four points. (1) Operations: n = 1,000 needs exactly 1,000 additions; the original would need a 210-digit number of calls. At n = 30 it is 30 against 2,692,537. (2)

Time: 0.054 ms for n = 1,000 (mean of 5 runs), below the 1 ms target. (3) The loop ran n = 1,000 and n = 200,000 with the default recursion limit; the memoised version failed from n = 500. (4) Memory: peak 0.5 KiB at n = 1,000, under the 1 KiB target, and it only grew from 0.2 to 0.7 KiB for n from 100 to 2,000 (that growth is the big answer number itself, not the algorithm).

- Size of the improvement: at n = 30 the loop used 89,751 times fewer operations and was 134,331 times faster; at n = 32 it was 220,286 times fewer operations and 312,836 times faster. The gap grows by a factor of 1.618 for every extra step. The operation counts are exact and repeatable; the times depend on my computer.
- New costs: (a) the code does not mirror the recurrence, so it is less obvious that it is right. I handled that by checking it against the recursive versions by hand for n = 1 to 6, by program for n = 10 to 32, and against memo at n = 100, 400, 1000 and 2000; (b) no cache between calls; (c) the time is not truly linear for huge n (Table Q7-4) because Python integers get longer. For n = 1,000 this does not matter (0.05 ms).
- Where the original may still be preferable: for very small n. At n = 10 the original takes 12 microseconds, which nobody would notice, and its code is the direct translation of the recurrence, so it is the best way to explain the problem and the best reference to test other versions against (I used it that way). It stops being acceptable around n = 30 (0.2 s) and is useless by n = 50 or so. The memoised version is a middle option: the same n + 1 calls as the loop and still readable, but it fails at n = 500 on default settings and needs about 700 times more memory at n = 1,000 than the loop (364.3 KiB against 0.5 KiB).
- Conclusion: the redesign is justified by evidence, not by assumption. The bottleneck was repeated work (99.9988 percent of the calls at n = 30), the objective was measurable, and the loop met every number in it. I did not just swap in a built-in function; I changed how the values are produced (bottom-up, two values) and tested it against the original on the same inputs.